CMDBUF RD
main_phase 过程式 TC

2026-11-11 第 32 份说明 上升沿后赋值 if 条件 Make 操作

当前 TC 按用户要求采用过程式写法：在 main_phase 中等待 @(posedge clk)，每个上升沿后执行一次带 if 条件的普通赋值。输入变量用 = 赋值；当前 TC 不使用 rand、constraint 或 randomize。模板已经写好每个输入、条件分支、时钟复位和执行接口，用户主要修改分支内的值。

main_phase 是当前轻量环境中的普通虚任务，不依赖 UVM。driver 将 TC 的值原样送到端口；运行时检查和独立采样负责发现实际违反协议或赋值时序的情况。未赋值的变量保持原值，不会自动随机；随机性需要显式调用 $urandom_range 或 $urandom。

1 创建并运行自己的 TC

```
cd /root/cmdbuf_rd_vcs_eda
make new-case CASE=my_read
# 编辑 workspace/simulation/vcs/tc/my_read.sv
make compile
make run CASE=my_read SEED=17
make wave CASE=my_read SEED=17
make coverage
```

Windows 在本地项目根目录将 make 换成 .\make.cmd。新建命令会生成本地 TC 并同步到 VM；本地修改后再执行 .\make.cmd sync。VM 与本地同时修改同一文件时，同步检测到冲突会停止。

TC 文件	可直接参考的内容
tc_directed.sv	完整入门模板，cycle=3 时赋值发送一条请求
tc_user_example.sv	用 if 在 cycle=3/4/8 给三个 XAU 发送请求
tc_random.sv	在合法 if 分支内显式生成随机输入
tc_retry.sv	用 if 分阶段开启 grant，验证失败和重试
tc_backpressure.sv	延迟 rready，形成背压和满载
tc_same_id.sv	相同 ID/地址的多条事务，独立 FIFO 检查
tc_banks.sv	覆盖 8 个 Bank 的两个 half
tc_reset.sv	改变周期与占空比，暂停时钟，复位取消与恢复

新增文件会自动加入 VCS 编译清单。Make 的 CYCLES 可以覆盖 TC 构造函数中的 num_cycles；默认回归仍是一条命令运行多个种子。

2 完整 main_phase 模板

下面是当前 tc_directed.sv 的有效代码。用户修改 = 右侧的值和 cycle 条件即可；保留已经写好的等待、上下文采样和驱动调用。

```
class tc_directed extends cmdbuf_main_tc;
    function new(); num_cycles=64; endfunction
    virtual task main_phase;
        for (int unsigned cycle=0; cycle<num_cycles; cycle++) begin
            @(posedge clk);
            #1;
            begin_cycle(cycle);
            rst_n = (cycle >= 3);
            rready = 3'b111;
            clk_low_ns = 5; clk_high_ns = 5; clk_pause_ns = 0;
            drive_control();
            #1;
            sample_context();
            if (!rst_n || arready == 3'b000)
                arvalid = 0;
            else if (cycle == 3)
                arvalid = 1;
            else
                arvalid = 0;
            if (arvalid) begin
                src = 0;
                arresp = 0;
            end else begin
                src = 0; arresp = 0;
            end
            araddr = 18'h12001;
            arid = 8'h2a; ar_port_id = 5'd6;
            ar_rd_gnt = 8'hff; rw_rd_gnt = 8'hff;
            for (int b=0; b<8; b++) begin
                if (req[b] && ar_rd_gnt[b] && rw_rd_gnt[b])
                    rd_mem_status[b] = (2'b01 << arb_addr[0]);
                else
                    rd_mem_status[b] = 0;
            end
            issue_data = 128'h0123456789abcdef_fedcba9876543210;
            issue_error = 0;
            if (mem_due) begin
                mem_rdata = due_data;
                pc_err = due_error;
            end else begin
                mem_rdata = 0; pc_err = 0;
            end
            drive_inputs();
        end
    endtask
endclass
```

例如改成 XAU2、地址 0x12345、ID 0x37，只需把有效分支中的 src 改为 2，并修改 araddr = 18'h12345、arid = 8'h37。条件判断仍使用 ==，过程赋值使用 =。

3 每个上升沿后的执行顺序

第 N 个上升沿采样上一组输入。本次 main_phase 在该沿之后开始准备新输入，新输入由第 N+1 个上升沿采样。#1 使用 testbench 的 1 ns 时间单位，防止在 DUT 正在采样或做 NBA 更新时抢写输入。

时刻	执行内容
posedge clk	DUT 与独立监视器采样上一组输入
上升沿后 1 ns	begin_cycle 记录本轮；给 rst_n、rready 和后续时钟参数赋值，调用 drive_control
上升沿后 2 ns	sample_context 读取稳定的 aready/req/arb_addr/MEM 历史；按 if 给其余输入赋值并 drive_inputs
下一个 posedge clk	采样刚才赋出的整组输入；监视器和原生 SVA 检查行为

3 1 真实运行的时间对应

cycle	等待到的沿 ns	赋值完成前的记录 ns	下一采样沿 ns
0	5	7	15
3	35	37	45

环境先建立已知复位并产生一个启动上升沿，再进入上述循环。cycle 是赋值编号，从 0 开始；默认 cycle=0/1/2 的三组赋值保持复位，cycle=3 解除复位并准备第一条请求。CYCLES=2000 表示正文赋值 2000 次，其中 1997 次是正常激励；启动沿、排空和静默阶段分别记录。

3 2 时钟暂停和复位仍在 TC 中设置

```
rst_n = (cycle >= 3 && cycle != 8);
clk_low_ns = 7; clk_high_ns = 3;
if (cycle == 8) clk_pause_ns = 40;
else clk_pause_ns = 0;
if (cycle >= 20) rready = 7;
else rready = 0;
```

low/high 设置用于后续完整脉冲，当前高电平按已经锁存的时长结束。clk_pause_ns 延长下一段低电平，随后自动恢复，main_phase 仍只在真实上升沿之后执行。low 和 high 的最小值均为 3 ns，保证驱动和反馈稳定有足够时间。

复位在 drive_control 时生效，可取消在途事务。MEM N+4 按真实上升沿计数，暂停期间不推进。原生 SVA 的 disable iff (!rst_n) 与源码保持原样；启动初始化和复位取消由仿真环境单独记录，没有新增复位序列 assume。

4 用 if 生成合法随机输入

定向 TC 在分支内填常量；随机 TC 在同一个合法分支内生成随机数。例如 ready 全零时直接赋 0，有接收能力时才随机决定是否发请求：

```
if (!rst_n || arready == 3'b000)
    arvalid = 0;
else
    arvalid = $urandom_range(1,0);

if (arvalid) begin
    src = $urandom_range(2,0);
    if ($urandom_range(1,0)) arresp = 3;
    else arresp = 0;
end else begin
    src = $urandom_range(3,0);
    arresp = $urandom_range(3,0);
end
```

Bank 状态也逐位用 if 处理：如果 req[b]、两路 grant 和 half 选择都命中，直接赋 busy=1；其他位可以随机。MEM 有返回任务时赋 due_data/due_error，否则可以随机背景值。数据在 TC 中生成，框架不会在复制时再作 valid 掩码或 busy 补位。

检查点	违反后的标识
有效 src 不在 0/1/2	MAIN_E03_SRC
有效 arresp 不在 0/3	MAIN_E04_ARRESP
ready 全零仍 valid=1	MAIN_E05_ZERO_READY
成功 ISSUE 的半区不 busy	MAIN_E06_SUCCESS_BUSY
MEM 返回与 N+4 历史不符	MAIN_T_MEM4
输入含 X/Z	MAIN_P_KNOWN
漏赋整拍或重复驱动	MAIN_MISSING_ASSIGNMENT / MAIN_DRIVE_ORDER
沿上抢写或反馈未稳定	MAIN_ONCE_AFTER_POSEDGE / MAIN_CONTEXT_TIMING

这些检查针对实际触发的违规。用户可以在分支内写自己的条件，也可以保留某个变量的旧值；环境不能凭一次仿真判断所有未触发路径都正确。当前种子固定后，过程式随机序列可重放，但不承诺对全部合法完整轨迹均匀抽样。

有效请求的所选 src 若没有 ready，这次尝试不会被接收，也不会自动重试。若需要重复尝试，在 main_phase 中明确写后续 cycle 的赋值。预约返回数据用 issue_data/issue_error；它们在实际 ISSUE 成功时记录，供 mem_due 分支驱动。

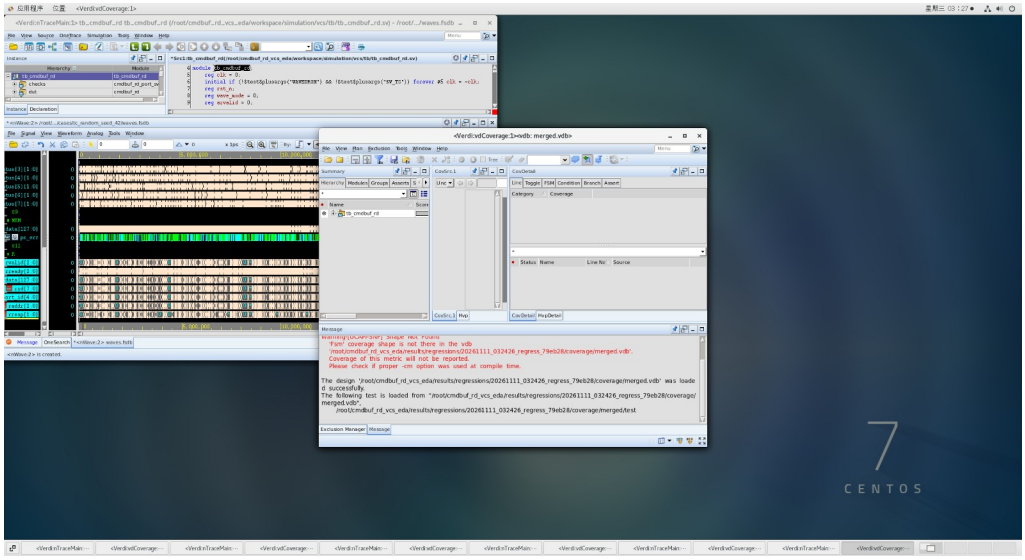
5 回归结果和定位入口

```
make regress JOBS=2
make regress RANDOM_ONLY=1 SEEDS=1,2,42 CYCLES=2000 JOBS=2
make new-case CASE=my_random TEMPLATE=tc_random
make regress RANDOM_ONLY=1 RANDOM_CASE=my_random SEEDS=1,2,42
make wave CASE=tc_random SEED=42
make coverage
make replay CASE=tc_random SEED=42 REGRESSION=批次号
make tc-test JOBS=2
```

本轮默认回归 16/16 通过：6 个定向 TC 和 10 个随机种子。随机正文共 20,000 次赋值，含 30 次复位和 19,970 次正常随机激励。连同排空及静默阶段，20,198 组赋值与下一采样沿的真实输入逐项一致；正常随机阶段 ready 全零 10,220 次，valid 全为 0，16 个 Bank/half 前提全部命中。

专项验收的 10 个合法、12 个预期失败用例符合预期，8 类独立审计篡改全部检出。旧约束套件的 40 个合法和 32 个预期失败用例、23 个 VM 单元测试也通过。新过程式种子 42 与旧声明式归档用例均已重放，赋值或求解记录、真实采样、事件及配置哈希一致。

覆盖率仅合并正常通过的用例：行 89.47%、条件 89.66%、翻转 85.32%、分支 75.61%；FSM 无可报告项。本轮波形与合并覆盖率已由 Make 在 VM Verdi 中加载。



每例保存 case.sv、assigned.csv、samples.csv、events.csv、FSDB、VDB 和错误上下文。assigned.csv 记录 TC 准备的值与发起时间，samples.csv 由独立上升沿进程记录实际端口，二者逐拍比对；make report、make trace 和 make wave 可定位问题。

当前构建 cc06ff8bfb2ea29a，回归 20261111_032426_regress_79eb28。111 个源码文件与 VM 构建一致。RTL、原生 SVA、规格和六轮历史签名未改。结论限当前登记协议的有限仿真；新协议规则需要同步更新 TC 分支、检查和验收用例。